

MAGNA · SECURITY REVIEW

Magna

Voting Escrow Updates.

A security review of Voting Escrow Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Magna	Voting Escrow Updates	Guardian	April 8 – May 1, 2024	May 8, 2024

§ 00

Contents.

39 pages — overview, confidence ranking, scope, 26 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 18 findings	P. 09
	Remediation Review · 8 findings	P. 29
§ 05	Disclaimer	P. 38
§ 06	About Guardian	P. 39

§ 01

Overview.

Guardian reviewed Voting Escrow Updates across 2 review rounds. The final report includes 7 High, 5 Medium, 14 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	magna-eng/merklevester
CLIENT	METHODOLOGY
Magna	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Voting Escrow Updates	3501d02eb1fa
PREPARED BY	
Guardian	
AUDIT WINDOW	
April 8 – May 1, 2024	
FINAL REPORT	
May 8, 2024	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	0	0	0	0	0
■ High	7	0	4	0	3
■ Medium	5	0	3	0	2
■ Low	14	0	3	0	11
■ Informational	0	0	0	0	0
All severities	26	0	10	0	16

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Voting Escrow Updates in `magna-eng/merklevester`. The scope includes 333 source lines across 4 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/MerkleVester.sol	193	313
src/MerkleValidator.sol	10	14
src/EscrowVester.sol	94	134
src/DelegationEscrow.sol	36	61

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Withdraw Not Possible Without Prior Delegation	Logical Error	HIGH	RESOLVED	p. 9
H-02	Revoked Allocations Are Stuck If Revoke All Feature Is Disabled	Logical Error	HIGH	RESOLVED	p. 11
H-03	Delegations Are Still Possible After Revoke Or Cancel	Logical Error	HIGH	RESOLVED	p. 12
M-01	Benefactor May Steal Delegation Reward In Other Tokens	Gaming	MEDIUM	RESOLVED	p. 14
M-02	Lack Of Support For Certain Rebasing Tokens	Logical Error	MEDIUM	RESOLVED	p. 15
M-03	Anyone Can Reduce Beneficiary's Voting Power	Logical Error	MEDIUM	ACKNOWLEDGED	p. 16
L-01	Redundant Fully Unlocked Validation	Optimization	LOW	RESOLVED	p. 17
L-02	totalEscrowed Storage Variable Re-assigned In A Loop	Optimization	LOW	RESOLVED	p. 18
L-03	Withdraw Function Unnecessarily Undelegates For A user	Unexpected Behavior	LOW	RESOLVED	p. 19
L-04	Modular Blacklist Suggestion	Suggestion	LOW	ACKNOWLEDGED	p. 20
L-05	Memory Variable Declared In For-loop	Optimization	LOW	RESOLVED	p. 21

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	totalEscrowed and escrowToken Are Private Variables	Suggestion	LOW	RESOLVED	p. 22
L-07	Lacking Events	Events	LOW	RESOLVED	p. 23
L-08	adminWithdrawFeature Not Declared Immutable	Optimization	LOW	RESOLVED	p. 24
L-09	Multisig Warning	Warning	LOW	RESOLVED	p. 25
L-10	Risk Of Selector Collision	Warning	LOW	RESOLVED	p. 26
L-11	Yield Can Be Withdrawn After Revoke	Warning	LOW	RESOLVED	p. 27
L-12	Invalid Withdrawal Invariant Check	Logical Error	LOW	RESOLVED	p. 28

REMEDIATION REVIEW

H-01	Funds Are Reclaimed Twice In WithdrawInterval	Logical Error	HIGH	ACKNOWLEDGED	p. 29
H-02	reclaimToFundWithdraw Incorrectly Implemented	Logical Error	HIGH	ACKNOWLEDGED	p. 30
H-03	Revoked Allocations Are Stuck	Logical Error	HIGH	ACKNOWLEDGED	p. 32
H-04	revokeAll Allows Revoked Vests To Claim In The Future	Logical Error	HIGH	ACKNOWLEDGED	p. 33
M-01	Users May Not Be Able To Delegate	Logical Error	MEDIUM	ACKNOWLEDGED	p. 34
M-02	Requested Withdrawal Amount Is Not Adjusted	Logical Error	MEDIUM	ACKNOWLEDGED	p. 35
L-01	Invariants For Withdrawal Functions	Validation	LOW	ACKNOWLEDGED	p. 36
L-02	Temporary Locking Of Revoked Funds	Logical Error	LOW	ACKNOWLEDGED	p. 37

HIGH **H-01**

RESOLVED

Withdraw Not Possible Without Prior Delegation

CATEGORY Logical Error
LOCATION MerkleVester.sol
REVIEW Main Review

DESCRIPTION

The `schedules` mapping inside `MerkleVester` is lazily stored as acknowledged by code comments. Therefore the withdrawal address within schedules is initially `address(0)` and is updated via the `_checkOrSetOriginalBeneficiary()` function. This function is used in three different places mainly:

1. During delegation related calls:

1. Delegation related calls (`delegateFunds()`, `reclaimDelegatedFunds()`, `withdrawEarned()`) calls `_validateEscrowRequest()` which in turn this function sets the withdrawal address in schedule before any operation correctly.

2. Changing beneficiary call:

1. `transferBeneficiaryAddress()` function also correctly sets the withdrawal address in schedule before changing it.

3. Withdrawal request:

1. `withdraw()` call continue with internal withdrawal calls according to the type of the vest. In both internal functions however (`_withdrawCalendar()`, and `_withdrawInterval()`) setting withdraw address comes after withdraw address check:

```
if ((schedules[calendar.allocation.id].withdrawalAddress != msg.sender) &&  
!adminWithdrawFeature) revert UnauthorizedWithdrawal();
```

Hence when `adminWithdrawFeature` is false, users cannot directly do withdrawals. In order to withdraw their funds in this scenario, the user has to either first delegate to escrow and then reclaim it so that their withdraw address will be recorded. However, if the escrow feature is not enabled, then the user cannot even delegate funds.

An admin can potentially transfer the beneficiary, but this is only possible if `transferableByAdmin` is enabled and unfavorable when there are numerous allocations. Ultimately, user withdrawals from the vester are prevented, a core functionality of the protocol.

H-01 Withdraw Not Possible Without Prior Delegation

CONTINUED

RECOMMENDATION

Inside `_withdrawCalendar` and `_withdrawInterval`, call function `_checkOrSetOriginalBeneficiary` prior to `if ((withdrawalAddress != msg.sender) && !adminWithdrawFeature) revert UnauthorizedWithdrawal();`

PROOF OF CONCEPT <https://github.com/GuardianAudits/Magna-Updates/pull/2>

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/56/files>

HIGH **H-02**

RESOLVED

Revoked Allocations Are Stuck If Revoke All Feature Is Disabled

CATEGORY Logical Error
LOCATION MerkleVester.sol
REVIEW Main Review

DESCRIPTION

The benefactor can `revoke` an allocation, such that funds are reclaimed and sent to the vester contract, and withdrawals for the allocation by the beneficiary are prevented. In the case that the `revokeAllFeature` is disabled, the benefactor will be unable to retrieve these funds, since both functions `defund` and `revokeAll` rely on the feature being enabled.

Function `rescueTokens` will revert since the token being vested cannot be rescued: `if (_errantTokenAddress == token) revert InvalidToken();`
Ultimately, revoked funds are stuck in the MerkleVester contract.

RECOMMENDATION

Consider removing the `revokeAllFeature` from the `defund` function entirely, or at minimum making a separate feature type for `defund` and documenting this behavior.

PROOF OF CONCEPT <https://github.com/GuardianAudits/Magna-Updates/pull/4>

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/58>

HIGH **H-03**

RESOLVED

Delegations Are Still Possible After Revoke Or Cancel

CATEGORY Logical Error
LOCATION MerkleVester.sol: 218
REVIEW Main Review

DESCRIPTION

When benefactor revoke's someone vesting, their `terminatedTimestamp` is changed so that they can not withdraw any funds (their withdrawable will be 0) as can be seen:

```
withdrawableschedules[allocationId].terminatedTimestamp = uint32(1);
```

The problem is that revoked users can still delegate the revoked funds. When the `revokeAllFeature` is enabled, the benefactor has the capability to remove specified amount of funds from vesting. The normal workflow for revocations can be assumed as: first revoke and then defund revoked amount.

When this happens, revoked users will have nothing as withdrawable and then the funds allocated to them initially will be removed from contract. However, since these users can still delegate funds that is recorded inside `Allocation` struct, they essentially will be able to delegate funds that do not belong to them which results in blocking other user's ability to delegate and withdraw because of insufficient funds in the contract.

Furthermore, allowing delegation after `revoke` or `cancel` can lead to funds stuck in the escrow contract forever. The allocation cannot be revoked or cancelled again to reclaim those funds as the `terminatedTimestamp` is non-zero. If `revokeAllFeature` is false (not ideal if true as all allocations will be revoked) and `transferableByAdmin` is false then benefactor can't reclaim those funds and they are stuck in the escrow contract.

RECOMMENDATION

Validate that the `terminatedTimestamp` for the allocation is zero inside function `delegateFunds`.

PROOF OF CONCEPT <https://github.com/GuardianAudits/Magna-Updates/pull/1>

H-03 Delegations Are Still Possible After Revoke Or Cancel

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/57/files>

MEDIUM **M-01**

RESOLVED

Benefactor May Steal Delegation Reward In Other Tokens

CATEGORY Gaming
LOCATION MerkleVester.sol: 248
REVIEW Main Review

DESCRIPTION

With the `rescueTokenFromEscrow` function the benefactor may withdraw delegation rewards from any delegator contract where tokens other than the delegation token have been rewarded.

However these rewards ought to belong to the beneficiary who delegated to receive those rewards, the benefactor should not be able to take them.

RECOMMENDATION

Consider allowing potential reward tokens to be blacklisted from the `rescueTokenFromEscrow` function. Otherwise modify the `rescueTokenFromEscrow` function such that only beneficiaries are able to rescue funds from their delegation contract.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/64>

MEDIUM **M-02**

RESOLVED

Lack Of Support For Certain Rebasing Tokens

CATEGORY Logical Error
LOCATION EscrowVester.sol
REVIEW Main Review

— DESCRIPTION

Certain rebasing tokens can not only increase in balance, but also decrease. For such tokens such as Ampleforth, the following validation in the EscrowVester could revert after rebases: `if (_escrowedFunds > IERC20(escrowToken).balanceOf(address(delegateAddress))) revert UnauthorizedDelegation();`

This can lead to a situation where a user delegates but after a rebase is unable to `_reclaimFunds`.

— RECOMMENDATION

Clearly document to users that rebasing tokens are not fully supported.

— RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/64>

MEDIUM **M-03**

ACKNOWLEDGED

Anyone Can Reduce Beneficiary's Voting Power

CATEGORY Logical Error
LOCATION MerkleVeter.sol: 285, 308
REVIEW Main Review

DESCRIPTION

Inside the internal withdraw functions, the following validation is performed:

```
if ((schedules[calendar.allocation.id].withdrawalAddress != msg.sender) &&  
!adminWithdrawFeature) revert UnauthorizedWithdrawal();
```

This check does the following:

- 1) If `adminWithdrawFeature` is false, only let `msg.sender` call `withdraw()`
- 2) If `adminWithdrawFeature` is true, don't revert.

While the purpose of the validation was to give permission only to the benefactor (admin) in the case that `adminWithdrawFeature` is enabled, it actually gives permission to everyone.

Since `withdraw` calls `reclaims` all funds from delegation escrow, the delegatee will lose all their voting power that comes from escrow. This prevents voting power from being delegated and maintained, which is a core functionality of the escrow updates. The withdrawals can be 1 wei and done continuously, as all funds are reclaimed from the escrow contract regardless of the requested withdrawal amount.

RECOMMENDATION

If `msg.sender` is not beneficiary but admin feature is enabled, perform a benefactor check.

PROOF OF CONCEPT <https://github.com/GuardianAudits/Magna-Updates/pull/3>

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Redundant Fully Unlocked Validation

CATEGORY Optimization
LOCATION MerkleVester.sol: 185
REVIEW Main Review

DESCRIPTION

In the `cancel` function the `getLeafJustAllocationDataWithFullyUnlockedCheck` function is used to perform validation on the fully unlocked timestamp.

However this function is invoked twice, when the validation from the first invocation is sufficient.

RECOMMENDATION

Remove the second invocation of the `getLeafJustAllocationDataWithFullyUnlockedCheck` function.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/59/files>

LOW **L-02**

RESOLVED

totalEscrowed Storage Variable Re-assigned In A Loop

CATEGORY Optimization
LOCATION EscrowVester.sol: 141
REVIEW Main Review

DESCRIPTION

In the `_revokeDelegated` function the `totalEscrowed` storage variable is decremented upon each iteration of the for-loop.

However it would be cheaper to cache the initial `totalEscrowed` value before beginning the loop and decrementing the cached value in the loop before finally re-assigning the `totalEscrowed` storage variable to the updated cached value.

RECOMMENDATION

Cache the initial `totalEscrowed` value before the for-loop, decrement the cached value on the stack in the for-loop, and assign the updated cached value to the storage variable after the for-loop.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/60/files>

LOW **L-03**

RESOLVED

Withdraw Function Unnecessarily Undelegates For A user

CATEGORY Unexpected Behavior
LOCATION MerkleVester.sol: 194, 314
REVIEW Main Review

— DESCRIPTION

When withdrawing, `MerkleVester.withdraw` function calls `EscrowVester._reclaimFunds` with 0 hardcoded for the amount out.

This will lead to all funds being undelegated from `DelegationEscrow` contract. If the amount being withdrawn is less than the total available, it is unnecessary to withdraw all of the tokens.

In some instances it is unnecessary to even call `EscrowVester._reclaimFunds` at all. This will force the user to call `MerkleVester.delegateFunds` again.

— RECOMMENDATION

Check the vested amount prior to calling `EscrowVester._reclaimFunds`. Then verify if the user is required to pull out the entirety of their funds from `DelegationEscrow` contract.

— RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/67>

LOW **L-04**

ACKNOWLEDGED

Modular Blacklist Suggestion

CATEGORY	Suggestion
LOCATION	EscrowVester.sol: 10
REVIEW	Main Review

— DESCRIPTION

In the EscrowVester contract the function selector blacklist does not allow benefactors to add additional selectors that are permanently blacklisted other than the ones hardcoded in the constructor.

Therefore if a benefactor wishes to use the MerkleVester escrow feature with a token that has a sensitive function that is not already included in the hardcoded blacklisted selectors, then the benefactor must assume some level of trust from the users that they will not whitelist the sensitive selector.

— RECOMMENDATION

Consider implementing functionality such that the benefactor may add additional blacklisted selectors upon construction.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Memory Variable Declared In For-loop

CATEGORY	Optimization
LOCATION	EscrowVester.sol
REVIEW	Main Review

DESCRIPTION

Declaring a memory variable inside a for loop is very gas intensive in Solidity. On each iteration a new memory variable is declared. The first 22 words of memory are priced linearly, but after that the price increases quadratically. This will significantly increase the chance of running out of gas in a transaction if the for loop iterates over a large array.

RECOMMENDATION

Declare the memory variable outside of the for loop and then reassign it in each iteration.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/61/files>

LOW **L-06**

RESOLVED

totalEscrowed and escrowToken Are Private Variables

CATEGORY	Suggestion
LOCATION	EscrowVester.sol: 34, 35
REVIEW	Main Review

DESCRIPTION

`totalEscrow` and `escrowToken` are private variables which will not have a getter function.

`totalEscrow` is kept track of in multiple locations costing gas from an sload and sstore. It currently tacks on unnecessary gas costs to transactions with no use case.

And `escrowToken` may be a useful queryable value.

RECOMMENDATION

Declare the `totalEscrowed` and `escrowToken` as public variables.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/62>

LOW **L-07**

RESOLVED

Lacking Events

CATEGORY	Events
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Events allow users to efficiently perform analysis of on-chain activity. Apps like Dune Analytics heavily rely on reading event logs. Adding events will allow users to gain insights on how the protocol is being utilized.

— RECOMMENDATION

Add events for major state changing functions.

— RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/68>

LOW **L-08**

RESOLVED

adminWithdrawFeature Not Declared Immutable

CATEGORY Optimization
LOCATION MerkleVester.sol: 28
REVIEW Main Review

DESCRIPTION

The `adminWithdrawFeature` is only assigned to in the constructor and cannot be updated, yet it is not declared as immutable.

RECOMMENDATION

Declare the `adminWithdrawFeature` as immutable.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/63>

LOW **L-09**

RESOLVED

Multisig Warning

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The `benefactor` role carries a lot of admin privileges that can lead to loss of users funds or loss of functionality. For example, the `benefactor` can prevent further delegations even if the `escrowFeature` is enabled, via calling `whitelistSelector` with `delegate(address)` set to false. The `benefactor` should be a multisig wallet to increase chances that the account is not compromised.

RECOMMENDATION

Be sure to recommend that customers use a multisig wallet for the `benefactor` role. Furthermore, clearly document admin privileges.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/64>

LOW **L-10**

RESOLVED

Risk Of Selector Collision

CATEGORY Warning
LOCATION EscrowVester.sol: 210
REVIEW Main Review

— DESCRIPTION

The EscrowVester whitelists certain functions to be called using their function selectors. However, multiple functions can produce the same selector, leading to a hash collision.

With the current whitelist, a user may pass arbitrary data to a token that doesn't call `delegate` but another function which happens to have the same selector, and lead to unexpected behavior.

— RECOMMENDATION

Clearly document this risk to users.

— RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/64>

LOW **L-11**

RESOLVED

Yield Can Be Withdrawn After Revoke

CATEGORY Warning
LOCATION MerkleVester.sol
REVIEW Main Review

DESCRIPTION

Even after a benefactor calls function `revokeAll`, the beneficiary can still withdraw earned yield through `withdrawEarned`. This may be unexpected if the the benefactor intended for the revoke to return absolutely all funds.

RECOMMENDATION

Clearly document this behavior to users.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/65>

LOW **L-12**

RESOLVED

Invalid Withdrawal Invariant Check

CATEGORY Logical Error
LOCATION IAirlockBase.sol: 99
REVIEW Main Review

DESCRIPTION

In the `_withdrawToBeneficiary` function the `_validateWithdrawalInvariants` function invocation passes the `withdrawableAmount` instead of the `requestedWithdrawalAmount` to verify the withdrawal invariants. This isn't entirely accurate as the `withdrawableAmount` does not necessarily match how much is being withdrawn, however there is no veritable impact.

RECOMMENDATION

Perform the `_validateWithdrawalInvariants` check on the `requestedWithdrawalAmount` rather than the entire `withdrawableAmount`. Otherwise simplify the invariant check to only do a post check that involves the contract balance afterwards.

RESOLUTION

Status: Resolved.

<https://github.com/magna-eng/merklevester/pull/70/files>

HIGH **H-01**

ACKNOWLEDGED

Funds Are Reclaimed Twice In WithdrawInterval

CATEGORY Logical Error
LOCATION MerkleVester.sol: 352
REVIEW Remediation Review

DESCRIPTION

In the `_withdrawInterval` function the original `_reclaimFunds` call remains, while an additional `_reclaimToFundWithdraw` call has been added.

RECOMMENDATION

Remove the original `_reclaimFunds` call in the `_withdrawInterval` function.

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

reclaimToFundWithdraw Incorrectly Implemented

CATEGORY Logical Error
LOCATION MerkleVester.sol: 395
REVIEW Remediation Review

DESCRIPTION

The `_reclaimToFundWithdraw` function is inappropriately implemented such that reclaims will occur when the `withdrawalAmount` can be sufficiently withdrawn from the non-escrowed portion of the user's allocation.

Additionally, when the unclaimed amount is greater than the escrowed amount escrowed tokens will not be reclaimed when they ought to be.

The clearest example is an entirely vested allocation, where the escrowed portion will not be reclaimed at all, but the entire token amount will still be claimed from the contract's balance. This user no longer has an allocation to vest and so therefore should not garner any voting power from escrowed tokens.

Additionally, the fact that the existing `_reclaimToFundWithdraw` implementation passes the written tests indicates a lack of coverage for the new behavior.

RECOMMENDATION

Implement new written test cases which add coverage for the new expected behavior surrounding the reclaiming of funds for withdrawal.

Additionally, use the suggested `_reclaimToFundWithdraw` function:

```
function _reclaimToFundWithdraw(uint256 withdrawalAmount, Allocation memory allocation,
uint256 withdrawn) internal {
    uint256 reclaim;
    // How much do we need to withdraw from escrow?
    uint256 leftToClaim = allocation.totalAllocation - withdrawn;
    uint256 allocationNotInEscrow = leftToClaim - escrowedFunds[allocation.id];
    if (allocationNotInEscrow < withdrawalAmount) {
        reclaim = withdrawalAmount - allocationNotInEscrow;
    }
}
```

H-02 reclaimToFundWithdraw Incorrectly Implemented

CONTINUED

```
uint256 contractBalanceWithReclaim = IERC20(token).balanceOf(address(this)) + reclaim;  
if (contractBalanceWithReclaim < withdrawalAmount) {  
    reclaim += withdrawalAmount - contractBalanceWithReclaim;  
}  
if (reclaim > 0) _reclaimFunds(reclaim, allocation.id);  
}
```

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Revoked Allocations Are Stuck

CATEGORY	Logical Error
LOCATION	MerkleVester.sol: 163
REVIEW	Remediation Review

DESCRIPTION

In the case that an allocation is cancelled partway through the vest, function `defund` is at risk of underflowing and preventing the terminated funds from being withdrawn. Consider the following example:

1. Bob creates 2 allocations for 50 tokens each, thus `totalKnownObligations = 100`.
2. Bob funds the Merkle contract for 100 tokens.
3. Alice and Chris delegate 50 tokens each, and merkle vester balance is now 0.
4. Bob cancels Alice's allocations halfway through, thus `totalTerminated = 25`.
5. `contractBalance = 50` since 50 tokens are reclaimed from delegation escrow.
6. Bob calls `defund` which reverts since `undefundable = 100 - 25 - 0 = 75` but `contractBalance = 50` so `defundable = 50 - 75` underflows.

RECOMMENDATION

Remove the `defundable` and `undefundable` logic. Afterwards, add a `withdrawnFromTerminated` variable, and then only allow the `totalTerminated - withdrawnFromTerminated` to be withdrawn.

RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

revokeAll Allows Revoked Vests To Claim In The Future

CATEGORY Logical Error
LOCATION EscrowVester.sol: 138
REVIEW Remediation Review

DESCRIPTION

When `revokeAll()` is invoked, it undelegates all users' delegations and then transfers all tokens out of the vester smart contract.

If the Benefactor decides to create a new unlock schedule and fund the vester smart contract again, users from previous unlock schedules are still eligible to claim their tokens that were supposed to be revoked. This will lead to theft of tokens from users of the current unlock schedule.

RECOMMENDATION

Set `schedules[allocationId].terminatedTimestamp` to `uint32(1)` inside of `_revokeAllDelegated()`. This will prevent users from being able to withdraw and delegate revoked tokens, similarly to `cancel()` and `revoke()`.

Additionally update the `totalTerminated` by the terminated amount in the `revokeAll` function.

Alternatively, consider removing the `_revokeAllDelegated` and `revokeAll` functions and simply require the benefactor to revoke vests one at a time.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Users May Not Be Able To Delegate

CATEGORY Logical Error
LOCATION Global
REVIEW Remediation Review

DESCRIPTION

If a contract is not fully funded for the entirety of an unlock schedule, only a portion of users will be able to delegate their funds. `_validateEscrowRequest` allows a user to delegate their total allocation minus what they have withdrawn, subtracted by the amount they are currently escrowing. If `MerkleVester.sol` is not funded for the entirety of an unlock schedule, user's who delegate first, will be able to delegate an unfair proportion of tokens to users who delegated later.

Example:

Total allocation for all users: 1000

Alice's allocation: 500

Bob's allocation: 500

Contract balance: 500

Alice decides to delegate all of her allocation. The contract balance would be 0, since all of the funds will be in Alice's delegation escrow. The transaction will pass the validations because Alice's allocation is 500 and she has not withdrawn or escrowed yet. Now Bob is unable to delegate since all funds are stored in Alice's delegation escrow.

RECOMMENDATION

Since the users' total allocations are lazily stored, there is no efficient solution to verify that the contract is properly funded. Warn users to verify the contract is properly funded for the entirety of an unlock schedule. Additionally, we added a check inside our recommended revision for `_reclaimToFundWithdraw` to cover withdrawal debt obligations.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Requested Withdrawal Amount Is Not Adjusted

CATEGORY Logical Error
LOCATION IAirlockBase.sol: 81
REVIEW Remediation Review

DESCRIPTION

In the case that `(requestedWithdrawalAmount > withdrawableAmount)`, the attempted withdraw reverts which is unexpected as right below the `requestedWithdrawalAmount` is adjusted to be no more than the available withdrawable: `requestedWithdrawalAmount = _optional0rMax(requestedWithdrawalAmount, withdrawableAmount);`

Consequently, requested withdrawals that should be auto-adjusted and pass execution actually end up reverting.

RECOMMENDATION

Remove `if (requestedWithdrawalAmount > withdrawableAmount) revert InsufficientFunds();` since `_optional0rMax` is used to ensure this condition.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Invariants For Withdrawal Functions

CATEGORY	Validation
LOCATION	MerkleVester.sol: 335, 359
REVIEW	Remediation Review

DESCRIPTION

When an allocation is revoked or cancelled, all their escrowed funds are reclaimed and it is not possible to delegate those funds further. Recommendation given for **H-02** should be combined with invariant checks for internal withdrawal functions so that revoked or cancelled allocations won't try reclaiming from escrow when there is no need to.

RECOMMENDATION

Change `_withdrawInterval` and `_withdrawCalendar` functions implementation with following diff in their respected places.

```
+ if (schedules[allocationId].terminatedTimestamp == 1) {  
+   revert();  
+ }  
uint256 unclaimedAmount = vestedAmount - schedule.withdrawn;  
uint256 requested = _optionalOrMax(withdrawalAmount, unclaimedAmount);  
+ if (schedules[allocationId].terminatedTimestamp == 0) {  
+   _reclaimToFundWithdraw(requested, allocation, allocationId);  
+ }  
- _reclaimToFundWithdraw(requested, unclaimedAmount, allocationId);
```

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Temporary Locking Of Revoked Funds

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

Since `totalKnownObligations` is only updated when a user interacts with the protocol, calling `cancel()` or `revoke()` can cause `totalTerminated` to be greater than `totalKnownObligations`. This leads to a revert inside of `defund()` when calculating `unDefundable`:

```
uint256 unDefundable = totalKnownObligations - totalTerminated - totalWithdrawn;
```

This will cause funds to be locked inside of the vester contract, until enough new users have interacted with the vester contract and can increase `totalKnownObligations` to be greater than `totalTerminated`.

RECOMMENDATION

Inside `cancel()` and `revoke()`, check if `totalKnownObligations` has been accounted for for the user. If `totalKnownObligations` has not been set, increase `totalKnownObligations` to prevent a revert in `defund()`.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.